Using Single- and Multi-Task Transformers for Classifying Controlled Experiment Protocols in Software Engineering: Initial Investigation

Fernando S. Grande
Informatics Department
State University of Maringá (UEM)
Maringá, PR, Brazil
fsg0314@gmail.com

André F. R. Cordeiro
Informatics Department
State University of Maringá (UEM)
Maringá, PR, Brazil
cordeiroandrefelipe@gmail.com

Edson OliveiraJr
Informatics Department
State University of Maringá (UEM)
Maringá, PR, Brazil
edson@din.uem.br

ABSTRACT

Aim: This study investigates the structural and semantic challenges of automatically extracting methodological data from controlled experiment reports in Software Engineering (SE). **Objective:** We evaluate how the rhetorical heterogeneity and dense technical terminology of SE literature impact the performance of Transformer-based models. We compare Single-Task Learning (STL) and Multi-Task Learning (MTL) architectures, while exploring domain-specific pretraining (e.g., SciBERT) and LLM-based synthetic data augmentation to mitigate the scarcity of structured metadata in SE. **Method:** A corpus of 3,218 manually annotated methodological excerpts, distributed over 23 classes and drawn from 200 SE controlled experiment papers, was derived from a pre-existing systematic mapping study following Jedlitschka et al. guidelines, and expanded to 4,298 training instances through LLM-based augmentation. **Results:** Formulaic SE sections (Introduction and Conclusion) are reliably classified by STL models. However, experiment Planning represents a significant bottleneck due to high semantic density and the frequent conflation of distinct experimental concepts within single excerpts. Domain-specific models (SciBERT) achieved the best scores in these sections. Furthermore, naive MTL architectures showed no evidence of positive transfer, which we attribute to the drastic rhetorical shifts across sections. **Conclusion:** While excerpt-level classification is viable for triage, extracting complex experimental design data exposes inherent ambiguities in current SE reporting practices. This underscores the need for context-aware extraction tools and highlights critical areas for refining SE documentation guidelines.

KEYWORDS

Software Engineering Experimentation, NLP, Text Classification, Transformers, Multi-task Learning, Data Augmentation, Reproducibility

1 Introduction

Empirical Software Engineering (ESE) has evolved into a mature scientific discipline that relies on controlled experiments to investigate human, technical, and organizational aspects of software development [2, 21]. As a cornerstone of empirical research, controlled experiments provide evidence of cause-and-effect relationships, enabling the evaluation of techniques, tools, methods, and behavioral assumptions under controlled conditions [28]. Despite advances in methodological rigor, challenges related to transparency, reproducibility, and reporting quality persist.

A key difficulty lies in extracting methodological information from published studies. Experiment reports vary considerably in structure, terminology, and completeness [12], making evidence synthesis largely manual. Meta-analyses, replications, and open science initiatives often require careful identification of sentences describing hypotheses, experimental design, variables, data collection, and analysis procedures. For industry practitioners, this manual extraction is a significant barrier to adopting evidence-based practices, as synthesizing actionable insights from dense technical reports is highly time-consuming.

Natural Language Processing (NLP), particularly Transformer-based models [25], offers opportunities to automate this process. Prior work in information extraction [1, 23, 29] and text classification [10, 15, 17] shows that pretrained language models effectively capture semantic regularities. However, applications explicitly targeting empirical software engineering remain limited.

This study investigates whether Transformer-based classifiers can automatically identify methodological components of controlled experiment reports. We compare two learning paradigms: **Single-Task Learning (STL)**, in which each task is trained independently, and **Multi-Task Learning (MTL)**, in which tasks share a standard encoder with task-specific output heads. To mitigate data sparsity, we also employ data augmentation using large language models (LLMs), following recent advances in text augmentation [4, 5, 31].

Building on foundational work in MTL theory [3, 30], we examine whether methodological tasks derived from experiment reports, highly structured yet rhetorically heterogeneous, benefit from shared representations. The study relies on a manually annotated dataset constructed in accordance with established reporting guidelines [12], covering four standard report sections: Abstract, Introduction, Planning, and Conclusion.

The main contributions of this work are: (i) a Transformer-based framework for methodological excerpt classification in software engineering experiment reports; (ii) a comparative evaluation of STL and MTL under heterogeneous methodological tasks; (iii) an empirical analysis of knowledge transfer in MTL settings; (iv) the use of LLM-based data augmentation for low-resource scientific NLP; and (v) a discussion of implications for reproducibility, metadata extraction, and open science.

The remainder of this paper is organized as follows. Section 2 discusses related work and the context of empirical software engineering reporting. Section 3 details the methodology, including dataset construction, data augmentation, and model architectures.

Section 4 presents the empirical results. Section 5 discusses the findings and their implications for software engineering practices. Section 6 presents the threats to validity. Finally, Section 7 concludes the paper and outlines future research directions.

2 Related Work

The evolution of Empirical Software Engineering (ESE) heavily relies on controlled experiments to build reliable, evidence-based knowledge [14]. Wohlin et al. [28] formalize the experimental process into stages such as scoping, planning, operation, and analysis. To ensure the quality and reproducibility of these stages, Jedlitschka et al. [12] proposed comprehensive reporting guidelines, establishing a structured protocol for documenting aspects like hypotheses, variables, experimental units, and procedures.

Despite these established guidelines, challenges associated with reporting quality have been discussed for decades. Studies have identified that experimental protocols in SE are frequently reported in a dispersed, inconsistent, and unstructured manner [12, 24]. This lack of standardization creates a massive bottleneck for secondary studies. Fully characterizing an SE experiment often requires the manual extraction of dozens of metadata fields (e.g., approximately 84 distinct fields in comprehensive repositories). This prohibitive human effort severely hinders the construction of large-scale repositories and meta-analyses, highlighting the urgent need for automated extraction tools. More recent efforts, such as those proposed by Furtado et al. [6], encourage standardized documentation of experimental procedures, but the automated extraction of this methodological content remains underexplored.

At the intersection of Natural Language Processing (NLP) and SE, several studies have successfully examined tasks such as bug triage, commit classification, requirements processing, and code summarization [19, 20]. However, the automatic extraction of complex methodological structures from scientific SE publications is still limited. Research in scientific information extraction, such as event extraction [29], named entity identification [23], and domain-specific modeling [11], offers essential foundations. Yet, purpose-built models and datasets tailored to the dense, rhetorically heterogeneous text of empirical software engineering are largely absent.

To bridge this gap, data augmentation has emerged as a key strategy, particularly for addressing the scarcity and severe class imbalance of domain-specific SE datasets. Recent work leveraging Large Language Models (LLMs), particularly ChatGPT-based augmentation [4, 31], has demonstrated strong results in generating synthetic samples for low-resource NLP domains, including scientific text.

Overall, prior studies reveal a critical gap: while the SE community recognizes the urgent need for structured experimental reporting to foster reproducibility, the automated NLP tools required to synthesize this knowledge at scale do not yet exist. Our work attempts to narrow this gap by developing and evaluating Transformer-based models explicitly designed to navigate the structural challenges of classifying methodological excerpts in SE controlled experiments.

3 Methodology

This section describes the methodological procedures used to design, train, and evaluate Transformer-based models for classifying methodological excerpts in controlled experiments. Our methodology follows established practices in empirical software engineering [12, 28] and current standards in scientific NLP and text classification [17, 20].

3.1 Aim and Research Questions

The aim of this study is analyzing Transformer-based learning approaches for the automatic classification of methodological excerpts in controlled experiment reports, for the purpose of evaluating their effectiveness in handling the structural and semantic challenges of Software Engineering literature, with respect to classification performance, rhetorical heterogeneity, and data scarcity, from the viewpoint of empirical software engineering researchers and tool builders, in the context of reproducible and open empirical research in Software Engineering.

While NLP techniques have been widely evaluated in generic domains, the reporting of controlled experiments in Software Engineering presents unique structural challenges, such as semantic overlap and ambiguous terminology (e.g., mixing independent and dependent variables in the same sentence). Therefore, this study aims to understand how the specific characteristics of SE reports interact with classification models.

The aim is refined into the following research questions:

- **RQ1.** What are the most challenging methodological elements of Software Engineering controlled experiments (based on [12] guidelines) to extract automatically?
- **RQ2.** How does the rhetorical heterogeneity and technical vocabulary of Software Engineering experimental sections impact knowledge transfer in Multi-Task Learning architectures?
- **RQ3.** To what extent does a scientific-domain vocabulary (e.g., SciBERT) help in disambiguating complex Software Engineering experimental concepts compared to general-purpose models?
- **RQ4.** How can synthetic data augmentation mitigate the scarcity of structured metadata for specific methodological categories in empirical Software Engineering?

3.2 Dataset Construction and Annotation

The dataset construction pipeline (illustrated in Figure 1) was structured into sequential stages to ensure transparency and reproducibility, transitioning from a pre-existing systematic mapping spreadsheet to a balanced, augmented corpus of methodological excerpts.

3.2.1 Corpus Selection and Source Data. The foundational corpus was derived from a Systematic Mapping Study (SMS) by Furtado et al. [6], which filtered 1,039 retrieved studies down to a final selection of 211 primary studies. The complete catalog, containing manually extracted methodological characteristics, is publicly available [7]. This SMS also supported previous studies of our research group on the quality evaluation [8] and the formalization [26] of SPL experiments.

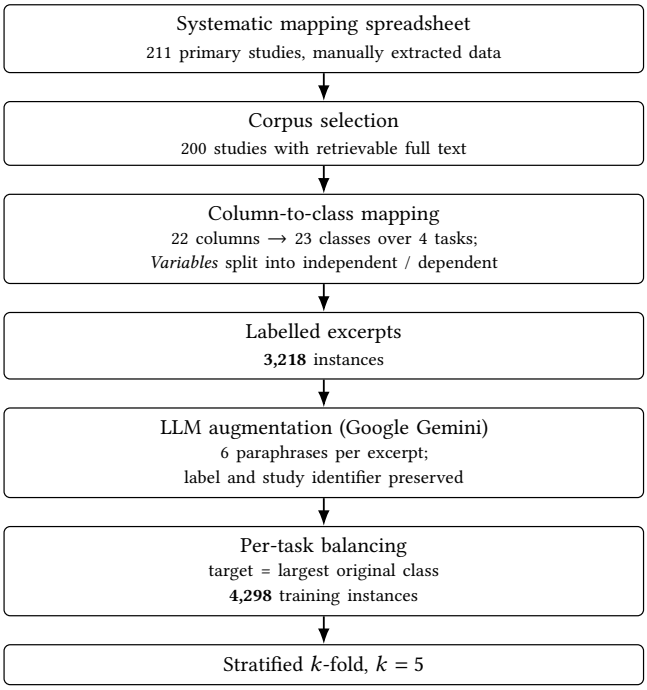


Figure 1: Dataset construction pipeline.

However, the methodological data recorded in this catalog often consisted of interpretative summaries provided by the human annotators. Because our objective is to train NLP models to automatically classify raw scientific text, we required the exact, literal excerpts as they originally appeared in the documents. Consequently, we needed to retrieve the full-text PDFs to anchor those interpretations to their original source sentences. During this retrieval phase, 11 of the 211 studies could not be downloaded or successfully processed and were therefore excluded. Thus, our final evaluated corpus consists of 200 peer-reviewed articles that report experimental procedures in SPL. With the raw full-text PDFs of these 200 studies successfully compiled, the subsequent challenge was to automatically map the human-interpreted spreadsheet annotations back to their precise textual origins within the documents.

3.2.2 Extractive Grounding and Class Mapping. To bridge this gap between the interpretative spreadsheet and the literal texts, we developed a custom LLM-driven extractive grounding pipeline using the Google Gemini API. Conceptually aligned with *extractive paraphrase span detection* [13], *extractive Question Answering* [22], and *LLM-based evidence extraction* [9], our approach treats each human-interpreted spreadsheet value as a semantic query. The model was prompted to locate and extract the exact semantic equivalent spans directly from the raw PDF text, strictly avoiding free generation. This schema-guided extraction enabled the dataset to be composed of verbatim excerpts directly linked to the original reports.

Once these literal excerpts were successfully retrieved, a subsequent parsing script systematically iterated over 22 methodological columns from the spreadsheet, mapping the extracted texts to their corresponding rhetorical class labels. Cells marked as unreported by the original annotators were discarded, and residual annotation markers were normalized. One column required additional treatment: while the original guidelines [12] define a single *Variables* construct, the annotators consistently recorded independent and dependent variables as separately marked segments. We split this column into two distinct classes (*Independent* and *Dependent Variables*), yielding a final label set of 23 classes across four tasks.

It is worth noting that the resulting 3,218 instances are annotator-delimited excerpts rather than strictly single sentences. While most excerpts consist of one or two sentences, sections such as *Summary* average 7.8 sentences per excerpt. The labels are therefore inherited from the manual data extraction of the SMS by Furtado et al. [6], which followed established guidelines for systematic mapping studies and whose extraction form is publicly available [7]. In that form, each of the 211 studies is described by fields that follow the report structure of Jedlitschka et al. [12] (e.g., Background, Goals, Hypotheses, Impact), and the class of each excerpt is the field from which it was taken (Table 1). No labels were assigned or revised in this work; our pipeline only located the text recorded in each field within the full text of the corresponding study.

3.2.3 Data Augmentation and Task Balancing. Because of this natural reporting variance, minority classes (such as *Impact* and *Limitations*) exhibited severe imbalance following the extraction phase. To mitigate this and stabilize the models' training, the LLM was instructed to generate six semantic paraphrases for each original excerpt, following validated augmentation practices for low-resource scientific domains [4, 31]. During this automated augmentation, the data pipeline strictly preserved both the inherited label and the identifier of the source study, ensuring that all six synthetic variations maintained the exact same rhetorical classification as their original counterpart and remained bound to the same document group.

After augmentation, each task was balanced by capping every class at the size of its largest original class, so that the 1:6 ratio raised minority classes towards the target without inflating already well-represented ones. This procedure brought most classes to approximately 200 instances. Two extreme classes remained below the target even after augmentation: *Limitations* reached 77 instances and *Impact* reached 42, since a sixfold expansion of a handful of original samples cannot span the gap. The full per-class distribution before and after augmentation, totaling 4,298 training instances, is reported in Table 1.

3.3 Model Architectures

To evaluate the feasibility of automated methodological extraction, two families of Transformer-based models were implemented: Single-Task Learning (STL) and Multi-Task Learning (MTL).

Single-Task Learning (STL) Models. In the STL paradigm, each classification task is modeled and trained independently. The conceptual premise is to build specialist models, where the entire network optimizes its weights solely for the semantic boundaries of a single domain task (e.g., distinguishing only planning elements). For each task, we fine-tuned a Transformer encoder followed by a task-specific classification head:

$$y = \text{softmax}(Wh_{[CLS]} + b) \tag{1}$$

Table 1: Per-class distribution before and after LLM-based augmentation.

Task	Class	Before	After
CT-Abstract	Background	196	198
	Objective	198	198
	Methods	192	198
	Results	154	198
	Limitations	11	77
	Conclusions	71	198
CT-Introduction	Problem Statement	197	200
	Research Objectives	197	200
	Context	200	200
CT-Planning	Goals	192	199
	Experimental Units	196	199
	Experimental Material	199	199
	Tasks	96	199
	Hypotheses	84	199
	Parameters	45	199
	Independent Variables	187†	199
	Dependent Variables		199
	Experiment Design	141	199
	Procedure	197	199
	Analysis Procedure	83	199
CT-Conclusion	Summary	200	200
	Impact	8	42
	Future Work	174	200
Total		**3,218**	**4,298**

†Recorded as a single *Variables* column in the source spreadsheet and split into two classes during extraction.

where $h_{[CLS]}$ is the final encoder embedding. We evaluated several base encoders: BERT-base, RoBERTa-base, DistilBERT, DeBERTa and SciBERT. The latter is particularly relevant due to its pretraining on scientific text domains, offering a vocabulary aligned with ESE jargon.

Multi-Task Learning (MTL) Models. MTL aims to improve generalization by leveraging domain-specific information contained in the training signals of related tasks. Theoretically, sharing representations across tasks acts as a regularizer, reducing overfitting and improving data efficiency, assuming positive transfer occurs.

To implement this, we utilized a *hard parameter sharing* parallel architecture. In this setup, the models share a foundational feature extractor, known as the *trunk* (the shared Transformer encoder), which is followed by task-specific layers, known as *branches* or *heads*. This means the MTL architecture uses one shared Transformer encoder simultaneously connected to four independent feedforward classification heads:

$$y_t = \text{softmax}(W_t h_{[CLS]} + b_t) \tag{2}$$

where $t \in \{1, \ldots, 4\}$ corresponds to each methodological label set (Abstract, Introduction, Planning, Conclusion). The theoretical intuition was that highly formulaic sections (like Abstracts) could provide stable gradient signals to help the shared encoder learn better representations for noisier and more complex sections (like Planning).

To address task imbalance and optimize the contribution of each task during training, we employed Uncertainty Weighting. Instead of using fixed weights, the multitask loss function dynamically learns the relative weights based on homoscedastic uncertainty:

$$\mathcal{L} = \sum_{t=1}^{4} \left(\frac{1}{2\sigma_t^2} \mathcal{L}_t + \log \sigma_t \right) \tag{3}$$

where $\mathcal{L}_t$ is the loss for task t, and σ_t is a learnable parameter representing the task's inherent noise (uncertainty). This mechanism allows the model to automatically down-weight tasks with higher uncertainty (noisier gradients) and up-weight simpler tasks, balancing the optimization process without manual tuning.

3.4 Training Procedure and Validation Strategy

All models were fine-tuned under identical experimental conditions to ensure comparability. The complete hyperparameter configuration was: 10 epochs, batch size 16, AdamW optimizer, learning rate 2×10^{-5}, dropout 0.1, weight decay 0.01, and warmup ratio 10%. Gradient clipping at 1.0 was used to stabilize training.

To evaluate the models' generalization capabilities, we employed a stratified 5-fold cross-validation strategy, and the same five folds were used for all STL and MTL models. The possible effect of synthetic paraphrases on the test folds is discussed in Section 6.

In each of the five iterations, 80% of the instances were used for training and 20% for testing. The model weights and optimizer states were re-initialized from scratch prior to each training cycle.

An ablation study (i.e., training the models exclusively on the original, non-augmented excerpts to isolate the augmentation effect) was not performed in this study and is left for future work.

Finally, to synthesize the models' overall effectiveness, performance is reported as the mean and standard deviation of the metrics across all five isolated folds.

3.5 Evaluation Metrics and Statistical Analysis

The primary metric was **Macro-F1**, due to class imbalance and multi-class structure. We additionally computed: Precision (macro-averaged), Recall (macro-averaged), Accuracy (micro-averaged), and Confusion matrices for all models.

Special attention was given to classes with limited representation (e.g., *Impact, Limitations*), assessing robustness under limited data.

Performance across folds was compared using **Welch's t-test** ($\alpha = 0.05$) [27]. This test was selected to evaluate whether the performance differences between STL and MTL approaches were statistically significant, without assuming equal variances. Because fold scores from k-fold cross-validation are not independent (training sets overlap, and STL and MTL were evaluated on the same folds), we also applied the variance correction for resampled t-tests proposed by Nadeau and Bengio [18]. Applied to the reported means and standard deviations, it gives p-values of 0.84 (CT-Abstract), 0.40 (CT-Introduction), 0.54 (CT-Planning) and 0.72 (CT-Conclusion). Additional paired tests on per-fold predictions are reported in the replication package and lead to the same conclusion.

4 Results

This section presents the empirical results from the evaluation of Transformer-based models for methodological excerpt classification in controlled-experiment reports. Results are reported for both Single-Task Learning (STL) and Multi-Task Learning (MTL) configurations. All values correspond to the mean and standard

Table 2: STL results (%) for all Single-Tasks.

Model	Accuracy	Precision	Recall	Macro F1
(Classification Task) CT-Abstract				
BERT	75.82 ± 1.34	78.88 ± 1.59	79.51 ± 1.96	78.93 ± 1.31
SciBERT	77.81 ± 1.09	81.15 ± 1.44	79.95 ± 1.54	80.12 ± 0.48
DistilBERT	73.94 ± 1.98	78.83 ± 1.14	77.31 ± 1.96	77.77 ± 0.95
RoBERTa	82.78 ± 1.10	80.75 ± 2.31	82.78 ± 1.72	81.48 ± 1.93
DeBERTa	76.49 ± 2.81	78.72 ± 2.18	82.00 ± 1.52	80.11 ± 1.65
CT-Introduction				
BERT	87.17 ± 1.55	88.11 ± 1.13	89.00 ± 1.63	88.22 ± 1.63
SciBERT	89.00 ± 1.22	90.01 ± 1.13	90.17 ± 1.62	89.84 ± 1.30
DistilBERT	89.00 ± 1.11	90.17 ± 1.02	89.50 ± 0.85	89.45 ± 0.89
RoBERTa	87.50 ± 1.39	88.19 ± 1.42	89.00 ± 1.43	88.30 ± 1.44
DeBERTa	86.67 ± 1.29	88.49 ± 1.66	87.67 ± 2.26	87.71 ± 1.53
CT-Planning				
BERT	60.82 ± 1.31	65.94 ± 2.12	62.15 ± 0.55	63.47 ± 1.10
SciBERT	62.01 ± 2.12	68.86 ± 2.18	64.50 ± 1.98	65.95 ± 1.86
DistilBERT	60.51 ± 2.13	64.98 ± 2.22	64.04 ± 1.19	63.96 ± 1.45
RoBERTa	61.19 ± 1.68	67.91 ± 2.84	63.44 ± 1.45	64.67 ± 2.09
DeBERTa	61.41 ± 1.04	66.73 ± 2.04	63.77 ± 1.84	64.12 ± 0.96
CT-Conclusion				
BERT	96.83 ± 1.32	97.02 ± 1.23	97.97 ± 1.32	97.42 ± 1.14
SciBERT	96.61 ± 1.87	96.83 ± 1.67	97.96 ± 1.11	97.33 ± 1.25
DistilBERT	97.06 ± 1.54	97.25 ± 1.49	98.19 ± 1.54	97.66 ± 1.44
RoBERTa	97.96 ± 1.10	98.13 ± 1.06	98.64 ± 1.31	98.31 ± 1.17
DeBERTa	95.70 ± 3.16	96.07 ± 2.86	98.20 ± 1.52	97.05 ± 1.96

deviation over the five stratified cross-validation folds. The primary evaluation metric is the Macro F1-score.

4.1 Single-Task Learning Results

Table 2 summarizes the performance of the STL models across the four classification tasks.

The results for the **CT-Abstract** classification task demonstrate moderate to high performance across the Single-Task Learning (STL) models. RoBERTa achieved the highest Macro F1-score (81.48 ± 1.93%) and Accuracy (82.78 ± 1.10%), closely followed by SciBERT, which yielded a highly consistent Macro F1-score with the lowest variance (80.12 ± 0.48%).

For the **CT-Introduction** task, all evaluated Transformer models exhibited strong predictive capabilities, with Macro F1-scores approaching or exceeding 88%. SciBERT emerged as the top-performing model, reaching a Macro F1-score of 89.84 ± 1.30% and an accuracy of 89.00 ± 1.22%, while DistilBERT offered highly competitive results with slightly lower standard deviations across most metrics.

The **CT-Planning** task proved to be the most challenging for the STL configurations, yielding the lowest overall metrics among all categories. In this task, SciBERT recorded the highest Macro F1-score (65.95 ± 1.86%) and Precision (68.86 ± 2.18%), whereas the standard BERT model yielded the lowest Macro F1-score at 63.47 ± 1.10%.

To provide a granular understanding of the classification bottlenecks, Table 3 details the class-level F1-Scores for the CT-Planning task. The transposed layout highlights the sharp contrast between highly recognizable parameters (e.g., Hypotheses) and ambiguous elements (e.g., Independent Variables).

Figure 2 plots the SciBERT column of Table 3, ordering the classes by F1-score to highlight the gap between well-recognized and ambiguous planning elements.

Table 3: F1-Score (%) per class for the CT-Planning task (STL Models).

Class	BERT	SciBERT	DistilBERT	RoBERTa	DeBERTa
Goals	59.1 ± 9.3	66.9 ± 12.5	67.9 ± 6.9	68.2 ± 9.3	73.8 ± 4.3
Experimental Units	52.8 ± 7.0	48.4 ± 5.8	47.5 ± 6.5	50.2 ± 9.9	52.3 ± 8.2
Experimental Material	55.6 ± 3.8	61.5 ± 13.3	62.5 ± 9.0	65.0 ± 10.8	62.4 ± 15.3
Tasks	61.0 ± 9.8	59.0 ± 8.2	59.1 ± 5.6	57.4 ± 4.5	55.9 ± 6.2
Hypotheses	93.1 ± 4.4	92.1 ± 3.9	94.2 ± 3.4	93.8 ± 4.7	91.1 ± 5.7
Parameters	91.6 ± 4.1	94.1 ± 2.8	94.3 ± 4.2	89.2 ± 4.1	88.9 ± 3.6
Dependent Variables	46.0 ± 4.3	52.6 ± 9.3	46.1 ± 8.9	56.9 ± 14.1	49.4 ± 16.8
Independent Variables	38.2 ± 8.1	36.0 ± 7.7	38.8 ± 3.3	31.5 ± 12.0	33.2 ± 12.2
Experiment Design	66.9 ± 7.6	67.6 ± 8.1	65.1 ± 5.4	52.3 ± 5.1	68.2 ± 4.7
Procedure	46.1 ± 11.5	52.8 ± 6.7	53.6 ± 15.5	55.8 ± 6.6	53.1 ± 7.9
Analysis Procedure	80.1 ± 8.3	84.8 ± 5.9	80.0 ± 5.2	82.9 ± 5.8	78.2 ± 5.3

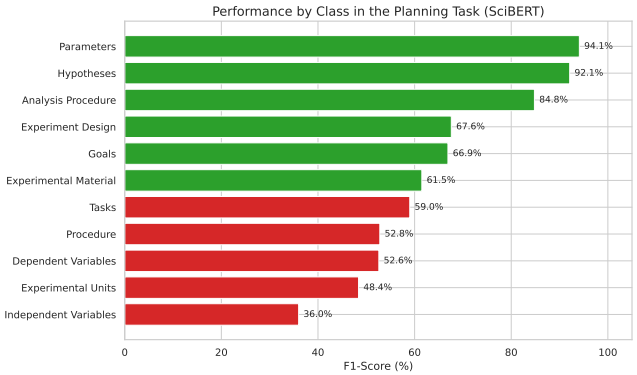


Figure 2: Performance by class in the CT-Planning task using the SciBERT model. Green bars indicate robust classification (≥ 60%), while red bars highlight ambiguous methodological elements.

Finally, the **CT-Conclusion** task results highlight highly accurate and stable classifications, with Macro F1-scores consistently above 97% for all models. RoBERTa achieved near-perfect performance in this category, registering the highest Macro F1-score of 98.31 ± 1.17% and an accuracy of 97.96 ± 1.10%. Table 4 shows high F1-scores for all classes in this section.

Table 4: F1-Score (%) per class for the CT-Conclusion task (STL Models).

Model	Summary	Impact	Future Work
BERT	97.55 ± 1.52	94.08 ± 0.45	97.99 ± 1.27
SciBERT	97.07 ± 1.22	93.28 ± 5.10	98.48 ± 0.95
DistilBERT	98.02 ± 2.15	93.26 ± 1.69	98.23 ± 1.30
RoBERTa	98.77 ± 1.09	92.77 ± 4.41	99.00 ± 0.94
DeBERTa	97.32 ± 2.36	88.82 ± 5.86	98.50 ± 1.45

4.2 Multi-Task Learning Results

Table 5 presents the results of the Multi-Task Learning (MTL) configuration. For each shared encoder, the table reports the macro-averaged F1 score obtained across the four tasks. All values correspond to the mean and standard deviation across five stratified cross-validation folds.

Table 5: MTL results (Macro F1-score %) across all tasks.

Shared Enc.	CT-Abstract	CT-Introduction	CT-Planning	CT-Conclusion
BERT	79.56 ± 0.95	87.74 ± 1.56	62.74 ± 1.12	97.89 ± 1.21
SciBERT	81.79 ± 1.06	88.80 ± 1.15	64.91 ± 1.50	97.32 ± 1.73
DistilBERT	78.58 ± 1.25	88.23 ± 0.86	63.41 ± 1.27	96.60 ± 2.12
RoBERTa	79.86 ± 2.86	88.11 ± 1.57	64.67 ± 1.56	97.31 ± 0.71
DeBERTa	79.22 ± 2.57	87.44 ± 3.69	63.29 ± 1.57	96.38 ± 2.16

4.3 Comparative Analysis: STL vs. MTL

This subsection presents a direct comparison between Single-Task Learning (STL) and Multi-Task Learning (MTL) approaches, aiming to assess the extent of learning transfer across tasks. The analysis considers the best-performing models from each approach, based on the Macro F1-score obtained for each task.

Table 6 summarizes the best results achieved per task, contrasting the performance of the best STL model with that of the best corresponding MTL model. Figure 3 presents the same values graphically to make the STL–MTL contrast across tasks easier to compare.

Table 6: Learning Transfer Analysis (STL vs. MTL) by Task (Macro F1-score in %).

Task	Best STL Model	Macro F1 (STL)	Macro F1 (MTL)	Transfer
CT-Abstract	RoBERTa	81.48	81.79 (SciBERT)	Positive
CT-Introduction	SciBERT	89.84	88.80 (SciBERT)	Negative
CT-Conclusion	RoBERTa	98.31	97.89 (BERT)	Negative
CT-Planning	SciBERT	65.95	64.91 (SciBERT)	Negative

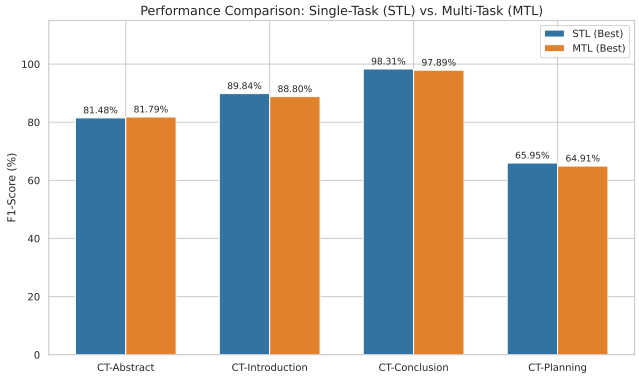


Figure 3: Performance comparison between the best Single-Task Learning (STL) and Multi-Task Learning (MTL) models across all classification tasks.

Figure 4 provides a complementary analysis based on the performance delta ($\Delta = F1_{MTL} - F1_{STL}$) for each base model and task through a heatmap visualization. Color intensity represents the magnitude and direction of performance change: positive values (green) indicate that the specific model benefited from the multitask learning setting for that task (positive transfer), while negative values (red) indicate negative transfer, and yellow indicates negligible change.

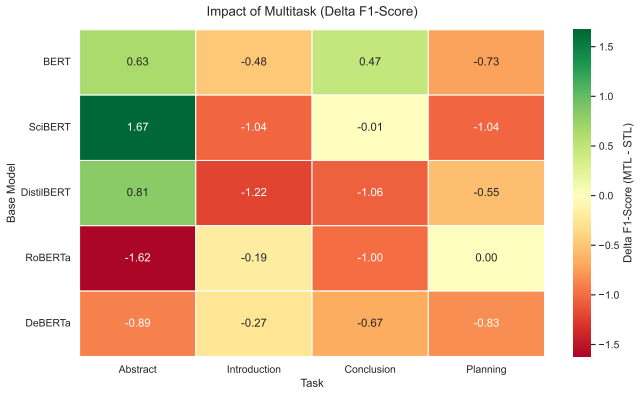


Figure 4: Heatmap of Multitask Learning Impact (Delta F1-score: $MTL - STL$)

To assess whether the observed differences between the STL and MTL approaches are statistically significant, Welch's t-test was applied [27]. Welch's test compares two means without assuming equal variances; since fold scores are not independent, the dependent-fold tests described in Section 3.5 are also reported.

The hypotheses tested were defined as follows:

- **Null hypothesis (H_0):** there is no statistically significant performance difference between STL and MTL.
- **Alternative hypothesis (H_1):** there is a statistically significant performance difference between STL and MTL.

The resulting p-values for each task are **CT-Abstract 0.76**, **CT-Introduction 0.22**, **CT-Planning 0.36**, and **CT-Conclusion 0.59**, which means H_0 could not be rejected. The dependent-fold tests (Section 3.5) led to the same result: no STL–MTL difference was significant in any task. Given only five folds, this indicates no evidence of a difference, not equivalence.

5 Discussion of Results

This section discusses the empirical findings by explicitly linking them to the research questions, with a focus on how the structural and semantic characteristics of SE experimental reports impact automated classification.

Challenging Methodological Elements in SE (RQ1). The results indicate that extracting methodological elements is highly dependent on the rhetorical structure of SE reports. Tasks associated with highly formulaic sections, such as CT-Conclusion and CT-Introduction, achieved strong and stable Macro F1-scores. In contrast, extracting elements from the experiment planning phase (CT-Planning) proved to be the most challenging bottleneck. Specifically, classes such as *Independent Variables* and *Experimental Units* yielded very low performance (e.g., 36.0% and 48.4% with SciBERT, respectively). This difficulty stems from the semantic density and

reporting habits in empirical SE. Authors frequently conflate distinct methodological concepts within a single excerpt, which often spans multiple sentences. For instance, the excerpt *"we conducted controlled experiments comparing PLUSS with MSVCM to assess the effort required by these approaches to derive and evolve SPL specifications"*, annotated as Goals, states both the treatments (PLUSS and MSVCM, the independent variable) and the response (effort, the dependent variable). This semantic overlap creates inherent ambiguity for excerpt-level classification, suggesting that fully automating the extraction of [12] guidelines requires broader context or highlights the urgent need for more standardized reporting practices in SE.

Furthermore, the CT-Planning task's complexity is exacerbated by the fact that distinct methodological concepts frequently co-exist within the same annotator-delimited block. Forcing a single-label classification in a domain where multiple entities natively co-occur loses this nuance and introduces ambiguity for the model. This indicates that moving from excerpt-level classification to token-level Named Entity Recognition (NER) may be needed to accurately extract dense planning metadata.

Rhetorical Heterogeneity and Multi-Task Learning (RQ2). Our empirical observations suggest that the rhetorical heterogeneity of SE reports poses a substantial challenge to knowledge transfer in Multi-Task Learning architectures. While a marginal increase in the absolute F1-score was observed for the CT-Abstract task, the MTL architecture yielded lower absolute scores for the remaining tasks, indicating a tendency toward negative transfer. Although the statistical tests revealed that these performance differences are not statistically significant, the absence of significant multitask performance gains highlights a core issue: the different sections of an SE experiment report serve drastically different rhetorical purposes. The Conclusion is typically synthetic and standardized, whereas the Planning section is technically dense, highly variable, and lacks a rigorous structural pattern across different SE papers. Forcing a shared encoder (hard parameter sharing) to learn across these highly heterogeneous texts may lead to conflicting gradients, degrading the representation of task-specific features.

Qualitatively inspecting the MTL misclassifications, we observed that the shared encoder often failed when general keywords appeared in dense technical contexts. For example, if a *Planning* excerpt contained the word "result" or "aim" while describing a variable, the MTL model frequently misclassified it as a *Conclusion* or *Introduction* excerpt. The shared representation prioritized these formulaic keywords over the specific methodological context, blurring the boundaries between sections. From a network optimization perspective, this negative transfer can be attributed to task conflict. The CT-Planning task is significantly more complex and noisy. Therefore, its gradients likely dominated the weight updates in the shared encoder, hindering the convergence of simpler tasks despite the use of Uncertainty Weighting. Additionally, the hard parameter sharing architecture may be too restrictive, as it assumes that low-level linguistic features are equally useful across all tasks.

Impact of Domain-Specific Vocabulary (RQ3). SciBERT presented an advantage in tasks involving dense methodological details, achieving the highest performance in CT-Planning (65.95%) and CT-Introduction (89.84%). This suggests that a scientific-domain vocabulary is beneficial for disambiguating the complex technical terminology and experimental designs frequently used in SE. However, for highly formulaic sections like the Conclusion, general-purpose models (e.g., RoBERTa) remained highly competitive, as these sections rely more on general linguistic regularities (e.g., "The results indicate...") than on specific technical jargon. This dichotomy highlights that the automated extraction of empirical protocols requires specialized approaches depending on the section. While robust, general-purpose models (like RoBERTa) are highly effective for sections with generalist rhetorical structures (Abstract and Conclusion), parsing tasks that demand a deeper comprehension of technical vocabulary and experimental design (Introduction and Planning) appear to benefit from scientific-domain models like SciBERT, although the differences between encoders are small (e.g., 89.84% vs. 89.45% for DistilBERT in CT-Introduction) and were not tested for significance.

Mitigating Metadata Scarcity via Data Augmentation (RQ4). The severe class imbalance observed in the original corpus reflects a broader challenge in empirical SE: the scarcity of structured methodological data to build repositories and support meta-analyses. The application of LLM-based data augmentation was intended to improve training stability and generalization. Following established LLM augmentation strategies [4, 31], we instructed the model to generate synthetic variations that preserved the semantic meaning while altering the lexical and syntactic structures.

To qualitatively interpret the success of this approach, we manually inspected the augmented samples. For instance, an original excerpt such as *"The participants used the Eclipse IDE..."* was synthetically paraphrased as *"The experimental treatment involved the utilization of the specific plugin..."*. This qualitative review suggests that the LLM diversified the vocabulary (e.g., replacing 'used' with 'involved the utilization') without altering the core methodological entity. The models' performance on minority classes is consistent with this observation. For instance, categories such as *Impact* and *Limitations*, which originally possessed only 8 and 11 samples respectively (and were expanded to only 42 and 77 instances), consistently achieved F1-scores exceeding 90% across most STL models. These results suggest that the synthetic data enriched the feature space of the minority classes. However, since the test folds may contain paraphrases of excerpts seen during training (Section 6) and no ablation without augmentation was performed, the effect of augmentation cannot be isolated, and the scores of these small classes may be optimistic. Synthetic data augmentation nonetheless appears to be a viable strategy to bootstrap the development of automated evidence synthesis tools in SE, mitigating the prohibitive human effort required to categorize experimental reports manually.

Table 7 summarizes the main insights from this discussion.

In our corpus of 200 controlled experiments, several elements recommended by the reporting guidelines are often missing: only 3% of the studies report the impact of the results, 5.5% state limitations in the abstract, 22% report parameters, and fewer than half report hypotheses (42%), analysis procedures (41%) or tasks (48%). Automatic extraction of these elements therefore supports software quality at the level of the research process: it can flag incomplete experiment reports against the guideline checklist, improve the traceability of evidence, and support replication, systematic reviews and meta-analyses.

Table 7: Summary of Key Discussion Points and Research Question Coverage

RQ(s)	Main Observations	Implications
RQ1	Performance varies substantially. Formulaic SE sections (Conclusion/Intro) are easily classified. Planning is the most challenging (e.g., Independent Variables) due to high semantic density and the SE authors' tendency to mix multiple experimental concepts in single sentences.	Excerpt-level classification is suitable for triage in SE, but extracting complex experimental design data exposes inconsistencies in current SE reporting practices, informing guideline refinement.
RQ2	Most tasks show negative, non-significant deltas under MTL. The drastic rhetorical shifts between SE sections (from standardized conclusions to variable planning descriptions) may cause shared encoders to struggle with conflicting signals.	Naive hard parameter sharing may be unsuitable for the heterogeneous nature of SE reports. Future tools may require section-aware MTL designs to avoid negative transfer.
RQ3	SciBERT achieves the highest scores in dense methodological tasks (Planning, Intro), suggesting that scientific embeddings help disambiguate technical SE experimental jargon. General models suffice for formulaic text.	Scientific-domain pretrained models may be preferable for mining SE experimental procedures and technical parameters.
RQ4	LLM-based augmentation was associated with high scores for underrepresented classes; its effect was not isolated by an ablation.	Synthetic augmentation is a promising strategy to overcome the severe lack of labeled empirical SE datasets for secondary studies.

Overall, while the primary focus of these automated extraction tools is to support academic research, they also offer significant practical applicability for the software industry. By automating the extraction of protocol-related and methodological data, practitioners and project managers can rapidly search, filter, and synthesize actionable evidence, such as which techniques or tools are effective in specific contexts, without the prohibitive effort of manually processing dense technical reports.

5.1 Qualitative Error Analysis

To better understand the limitations of excerpt-level classification, particularly in the challenging CT-Planning task, we structured a systematic error analysis pipeline consisting of a quantitative triage followed by a qualitative inspection.

First, the quantitative evaluation was conducted by logging every individual prediction from the test set across all cross-validation folds. By computing disaggregated metrics (Precision, Recall, and F1-score) for each specific sub-section using *scikit-learn*, we identified the exact boundaries where the SciBERT model struggled the most. The metrics revealed severe classification bottlenecks specifically within the *Independent Variables*, *Dependent Variables*, and *Experimental Units* classes.

Once these problematic classes were isolated quantitatively, we proceeded with a manual qualitative inspection to understand the underlying semantic causes of these errors. We extracted the raw text of the most frequent False Positives and False Negatives from the logs and conducted a semantic and syntactic analysis of the excerpts. This qualitative review revealed that the errors were not random, but were driven by high semantic overlap.

In empirical SE reports, authors rarely isolate methodological entities into distinct sentences. Instead, multiple concepts often coexist within the same syntactic structure. For instance, the excerpt *"we compare our approach, with respect to the required time to achieve the pairwise coverage, against existing sampling algorithms"*, annotated as Experimental Units, was classified as Goals by SciBERT: it names the treatments (our approach vs. existing sampling algorithms) and the response (time to achieve pairwise coverage) in the same sentence.

When the Transformer model processes this excerpt, the attention mechanism captures strong contextual signals for both variable types. However, because the task is framed as single-label excerpt classification, the model is forced to assign only one label, inevitably penalizing the metric for the other present class. This forced choice explains the low recall and precision quantitatively observed for Independent Variables (e.g., 36.0% F1-Score).

This error analysis reveals a critical methodological takeaway: while excerpt-level classification is adequate for rhetorical sections (Abstract, Conclusion), it lacks the granularity required for the dense technical descriptions found in Experiment Planning. Future automated extraction tools may benefit from moving from excerpt classification to token-level Named Entity Recognition (NER) to accurately capture the overlapping methodological constructs inherent to SE literature.

6 Threats to Validity

To ensure the evaluation of our findings, we categorize the threats to the validity of this study into four main aspects:

Internal Validity. The main internal threat stems from the annotation of the dataset. The labels come from the manual data extraction of the SMS by Furtado et al. [6, 7], whose extraction procedure and inter-annotator agreement were not reported; the excerpts were not re-annotated in this work. Label noise therefore cannot be quantified, and overlapping methodological roles in technically dense texts introduce inherent ambiguity. Additionally, technical artifacts from PDF text extraction and potential errors in locating the excerpts in the PDF text might affect the representations learned by the Transformer models.

Construct Validity. A primary threat to construct validity lies in modeling the information extraction problem as a excerpt-level text classification task. Methodological meaning often spans multiple paragraphs or, conversely, multiple concepts (e.g., independent and dependent variables) coexist within a single sentence. This forces a single-label abstraction that may not capture the true methodological nuance, suggesting that a NER approach might be more appropriate for future work. Furthermore, while LLM-based data augmentation was necessary to mitigate extreme class imbalance, the generated synthetic data could introduce subtle biases, hallucinations, or unrealistic reporting patterns that do not perfectly mirror human authors.

Additionally, the extractive grounding process relied on the LLM's unsupervised span detection. While the prompt strictly requested literal extractions, recent evaluations indicate that LLMs may occasionally hallucinate quotes that appear literal but deviate from the source text [16]. Because our pipeline did not implement an automated substring verification (trust-but-anchor) post-extraction, minor provenance discrepancies might exist within the training data. Finally, the classification units are annotator-delimited excerpts inherited from the original mapping study rather than sentences produced by an automatic segmenter. This preserves the boundaries defined in the original extraction, but it means that a single instance may span several sentences, which compounds the multi-concept ambiguity discussed above.

External Validity. The dataset relies entirely on English-language SE controlled experiments and is particularly focused on the SPL sub-domain. While SPL experiments follow standard SE protocols, this specific technical terminology and domain-specific reporting habits restrict the broad generalization of our models to other SE sub-domains, different empirical methods (e.g., case studies, surveys), or different publication cultures.

Conclusion Validity. The moderate overall dataset size and the severe initial class imbalance may increase the models' sensitivity to within-fold variance. Consequently, the performance differences observed between STL and MTL architectures must be interpreted cautiously. To mitigate this threat, we used stratified 5-fold cross-validation to keep class proportions similar across folds, and complemented Welch's t-test with tests that account for dependent folds (Section 3.5). No STL–MTL difference was significant; with five folds the power of these tests is low, so this indicates no evidence of positive transfer rather than equivalence.

Reproducibility. The replication package (see ARTIFACT AVAILABILITY) provides an independent re-implementation of the full pipeline, with fixed seeds, data hashes and all predictions. With folds strictly grouped by study, the re-implementation reproduces the CT-Introduction results and the main conclusion of this study (MTL does not significantly outperform STL in any task), but yields lower CT-Conclusion and CT-Abstract scores and higher CT-Planning scores than those reported here. Impact and Limitations have very few original excerpts (8 and 11), each expanded with synthetic paraphrases, so their scores depend strongly on how the paraphrases are distributed across folds: with folds strictly grouped by study, as in the re-implementation, they are much lower, and the values reported here should be read as optimistic.

7 Final Remarks

We presented a comparative analysis of STL and MTL Transformer models for classifying methodological excerpts in experiment reports. Although STL yielded higher absolute Macro F1-scores across most tasks, the statistical analysis failed to reject the null hypothesis, indicating no significant performance difference between the paradigms. Nevertheless, the absence of evidence of positive transfer suggests that naive hard parameter sharing may be insufficient to overcome the rhetorical heterogeneity of SE literature, making specialist models (STL) currently a simpler and more stable choice. These conclusions are therefore partial: with only five folds the statistical tests have low power, and future work should use repeated cross-validation and a larger corpus to test whether the observed differences are significant.

As detailed in Section 6, these conclusions must be interpreted within the context of the study's limitations. The reliance on a dataset exclusively from the SPL domain restricts immediate generalization to other SE areas. Furthermore, framing the problem as single-label excerpt classification could not resolve the overlapping nature of methodological concepts, and the use of LLM-generated synthetic data inherently introduces patterns that may differ from natural human scientific writing. Finally, the classifiers were not evaluated in practice by researchers performing systematic reviews or replications.

To address these limitations and appropriately guide future research efforts, we plan to expand the dataset across different venues, study types, and SE sub-domains to improve generalizability. We also plan an ablation study without augmentation to isolate its effect on minority classes, and a user study in which researchers apply the classifiers, integrated into a tool, to extract data from experiment reports. In addition, the misclassified excerpts will be examined with systematic qualitative methods, such as content analysis, to explain the error patterns observed in RQ2 and RQ4. To overcome the limits of excerpt-level classification, we propose the investigation of token-level NER techniques, which provide the granularity required to extract precise text fragments. Furthermore, investigating more advanced MTL architectures, such as soft parameter sharing, may mitigate the negative deltas observed. Ultimately, integrating these refined models into an automated software pipeline will support FAIR compliance, enabling the software industry to seamlessly extract actionable evidence from academic reports and apply evidence-based practices to real-world development environments.

ARTIFACT AVAILABILITY

The replication package containing the dataset, automated extraction script, model training pipeline, and full environment configuration is publicly available on Zenodo at: https://doi.org/10.5281/zenodo.22760606. Additionally, the source code repository is mirrored on GitHub at https://github.com/fernando-grande/sbqs2026-replication-package.

REFERENCES

[1] Mohd Hafizul Afifi Abdullah, Norshakirah Aziz, Said Jadid Abdulkadir, Hitham Seddig Alhassan Alhussian, and Noureen Talpur. 2023. Systematic Literature Review of Information Extraction From Textual Data: Recent Methods, Applications, Trends, and Challenges. *IEEE Access* 11 (2023), 10535–10562. doi:10.1109/ACCESS.2023.3240898

[2] Victor R. Basili. 1996. Editorial. *Empirical Software Engineering* 1, 2 (1996), 105–108. doi:10.1007/BF00368700

[3] Shijie Chen, Yu Zhang, and Qiang Yang. 2024. Multi-Task Learning in NLP: An Overview. *Comput. Surveys* 56, 12, Article 295 (2024). doi:10.1145/3663363

[4] Haixing Dai, Zhengliang Liu, Wenxiong Liao, Xiaoke Huang, Yihan Cao, Zihao Wu, Lin Zhao, Shaochen Xu, Fang Zeng, Wei Liu, Ninghao Liu, Sheng Li, Dajiang Zhu, Hongmin Cai, Lichao Sun, Quanzheng Li, Dinggang Shen, Tianming Liu, and Xiang Li. 2025. AugGPT: Leveraging ChatGPT for Text Data Augmentation. *IEEE Transactions on Big Data* 11, 3 (2025), 907–918. doi:10.1109/TBDATA.2025.3536934

[5] Steven Y. Feng, Varun Gangal, Jason Wei, Sarath Chandar, Soroush Vosoughi, Teruko Mitamura, and Eduard Hovy. 2021. A Survey of Data Augmentation Approaches for NLP. In *Findings of ACL-IJCNLP*. 968–988. doi:10.18653/v1/2021.findings-acl.84

[6] Viviane Furtado, Edson OliveiraJr, and Marcos Kalinowski. 2021. Guidelines for Promoting Software Product Line Experiments. In *SBCARS*. ACM, 31–40. https://doi.org/10.1145/3483899.3483909

[7] Viviane R. Furtado, Edson OliveiraJr, and Marcos Kalinowski. 2020. *Replication Package: Systematic Mapping Study on Software Product Line Experiments.* doi:10.5281/zenodo.3893302

[8] Viviane R. Furtado, Henrique Vignando, Victor França, and Edson OliveiraJr. 2019. Comparing Approaches for Quality Evaluation of Software Engineering Experiments: An Empirical Study on Software Product Line Experiments. *Journal of Computer Science* 15, 10 (2019), 1396–1429. doi:10.3844/jcssp.2019.1396.1429

[9] Gerald Gartlehner, Lisa Kahwati, Richard Hilscher, Ian Thomas, Shannon Kugley, Karen Crotty, Meera Viswanathan, Barbara Nussbaumer-Streit, Gary Booth, Nancy Erskine, Amy Konet, and Robert Chew. 2024. Data extraction for evidence synthesis using a large language model: A proof-of-concept study. *Research Synthesis Methods* 15, 4 (Jul 2024), 576–589. Epub 2024 Mar 3. doi:10.1002/jrsm.1710

[10] Andrea Gasparetto, Matteo Marcuzzo, Alessandro Zangari, and Andrea Albarelli. 2022. A Survey on Text Classification Algorithms. *Information* 13, 2 (2022). doi:10.3390/info13020083

[11] Archana Goyal, Vishal Gupta, and Manish Kumar. 2018. Recent Named Entity Recognition and Classification Techniques. *Computer Science Review* 29 (2018),

21–43. doi:10.1016/j.cosrev.2018.06.001

[12] Andreas Jedlitschka, Marcus Ciolkowski, and Dietmar Pfahl. 2008. *Reporting Experiments in Software Engineering*. Springer, 201–228. doi:10.1007/978-1-84800-044-5_8

[13] Jenna Kanerva, Henri Kitti, Li-Hsin Chang, et al. 2025. Semantic search as extractive paraphrase span detection. *Language Resources and Evaluation* 59 (2025), 257–276. doi:10.1007/s10579-023-09715-7

[14] Barbara Ann Kitchenham, David Budgen, and Pearl Brereton. 2015. *Evidence-Based Software Engineering and Systematic Reviews* (1 ed.). Chapman and Hall/CRC. doi:10.1201/b19467

[15] Kamran Kowsari, Kiana Meimandi, Mojtaba Heidarysafa, Sanjana Mendu, Laura Barnes, and Donald Brown. 2019. Text Classification Algorithms: A Survey. *Information* 10, 4 (2019). doi:10.3390/info10040150

[16] Malgorzata Lagisz, Ayumi Mizuno, Kyle Morrison, Pietro Pollo, Lorenzo Ricolfi, Yefeng Yang, and Shinichi Nakagawa. 2025. Using Elicit AI research assistant for data extraction in systematic reviews: a feasibility study across environmental and life sciences. *EcoEvoRxiv* (2025). doi:10.32942/x2f346

[17] Qian Li, Hao Peng, Jianxin Li, Congying Xia, Renyu Yang, Lichao Sun, Philip S. Yu, and Lifang He. 2022. A Survey on Text Classification. *ACM Transactions on Intelligent Systems and Technology* 13, 2 (2022). doi:10.1145/3495162

[18] Claude Nadeau and Yoshua Bengio. 2003. Inference for the Generalization Error. *Machine Learning* 52, 3 (2003), 239–281. doi:10.1023/A:1024068626366

[19] Daniel W. Otter, Julian R. Medina, and Jugal K. Kalita. 2021. A Survey of the Usages of Deep Learning for NLP. *IEEE Transactions on Neural Networks and Learning Systems* 32, 2 (2021), 604–624. doi:10.1109/TNNLS.2020.2979670

[20] Narendra Patwardhan, Stefano Marrone, and Carlo Sansone. 2023. Transformers in the Real World: A Survey on NLP Applications. *Information* 14, 4 (2023). doi:10.3390/info14040242

[21] Dewayne E. Perry, Adam A. Porter, and Lawrence G. Votta. 2000. Empirical Studies of Software Engineering: A Roadmap. In *Proceedings of The Future of Software Engineering*. ACM, New York, NY, 345–355. doi:10.1145/336512.336586

[22] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 2383–2392. doi:10.18653/v1/D16-1264

[23] Sunita Sarawagi. 2008. Information Extraction. *Foundations and Trends in Databases* 1, 3 (2008), 261–377. doi:10.1561/1900000003

[24] Forrest Shull, Janice Singer, and Dag I. K. Sjøberg. 2008. *Guide to Advanced Empirical Software Engineering*. Springer, Berlin, Heidelberg.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. arXiv:1706.03762 https://arxiv.org/abs/1706.03762

[26] Henrique Vignando, Viviane R. Furtado, Lucas O. Teixeira, and Edson OliveiraJr. 2020. OntoExper-SPL: An Ontology for Software Product Line Experiments. In *Proceedings of the 22nd International Conference on Enterprise Information Systems (ICEIS 2020)*, Vol. 2. SciTePress, 401–408. doi:10.5220/0009575404010408

[27] Bernard L Welch. 1947. The generalization of 'Student's' problem when several different population variances are involved. *Biometrika* 34, 1/2 (1947), 28–35.

[28] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer, Germany. doi:10.1007/978-3-642-29044-2

[29] Wei Xiang and Bang Wang. 2019. A Survey of Event Extraction From Text. *IEEE Access* 7 (2019), 173111–173137. doi:10.1109/ACCESS.2019.2956831

[30] Yu Zhang and Qiang Yang. 2022. A Survey on Multi-Task Learning. *IEEE Transactions on Knowledge and Data Engineering* 34, 12 (2022), 5586–5609. doi:10.1109/TKDE.2021.3070203

[31] Huanhuan Zhao, Haihua Chen, Thomas A. Ruggles, Yunhe Feng, Debjani Singh, and Hong-Jun Yoon. 2024. Improving Text Classification with Large Language Model-Based Data Augmentation. *Electronics* 13, 13 (2024). doi:10.3390/electronics13132535